

cann-samples

versus

cuda-samples

华为 **cann-samples** (分支 sync/upstream/master) 与英伟达 **NVIDIA/cuda-samples** (master · Release v13.3) 的目录级、定位级、覆盖级与开源生态级全方位对比分析。

- 目录对比
- 定位对比
- 覆盖范围
- 开源开放定位
- 开发者样例
- 生态治理

cann-samples gitcode.com/cann/cann-samples 厂商 华为 Huawei 平台 Ascend NPU (昇腾 910B/C · 950) 主托管 GitCode (GitHub 只读镜像) 内核 Ascend C .asc 许可 CANN OSL v2.0 受限 · 非 OSI	cuda-samples github.com/NVIDIA/cuda-samples 厂商 英伟达 NVIDIA 平台 CUDA GPU (含 Tegra / DriveOS) 主托管 GitHub (PR 原生协作) 内核 CUDA C++ .cu / .cuh 许可 BSD-3-Clause 宽松 · OSI
---	---

目录

00	执行摘要	一页看懂两仓分野
01	总览与加权能力画像	七轴雷达 · 头部指标
02	目录结构与定位映射	平铺 vs 深嵌套 · 类目对应
03	产品定位与开源开放定位	许可证逐条对照 · 记分卡
04	覆盖范围：广度 vs 深度	领域热力矩阵 · 类目分布
05	给开发者提供的样例	教学范式 · 构建门槛 · 独占能力
06	开源开放生态与治理	托管 · 贡献 · CI · 成熟度
07	结论与选型建议	场景化推荐矩阵
08	数据附录与核验日志	全量计数 · 对抗式验证

00 执行摘要

一页看懂两仓分野

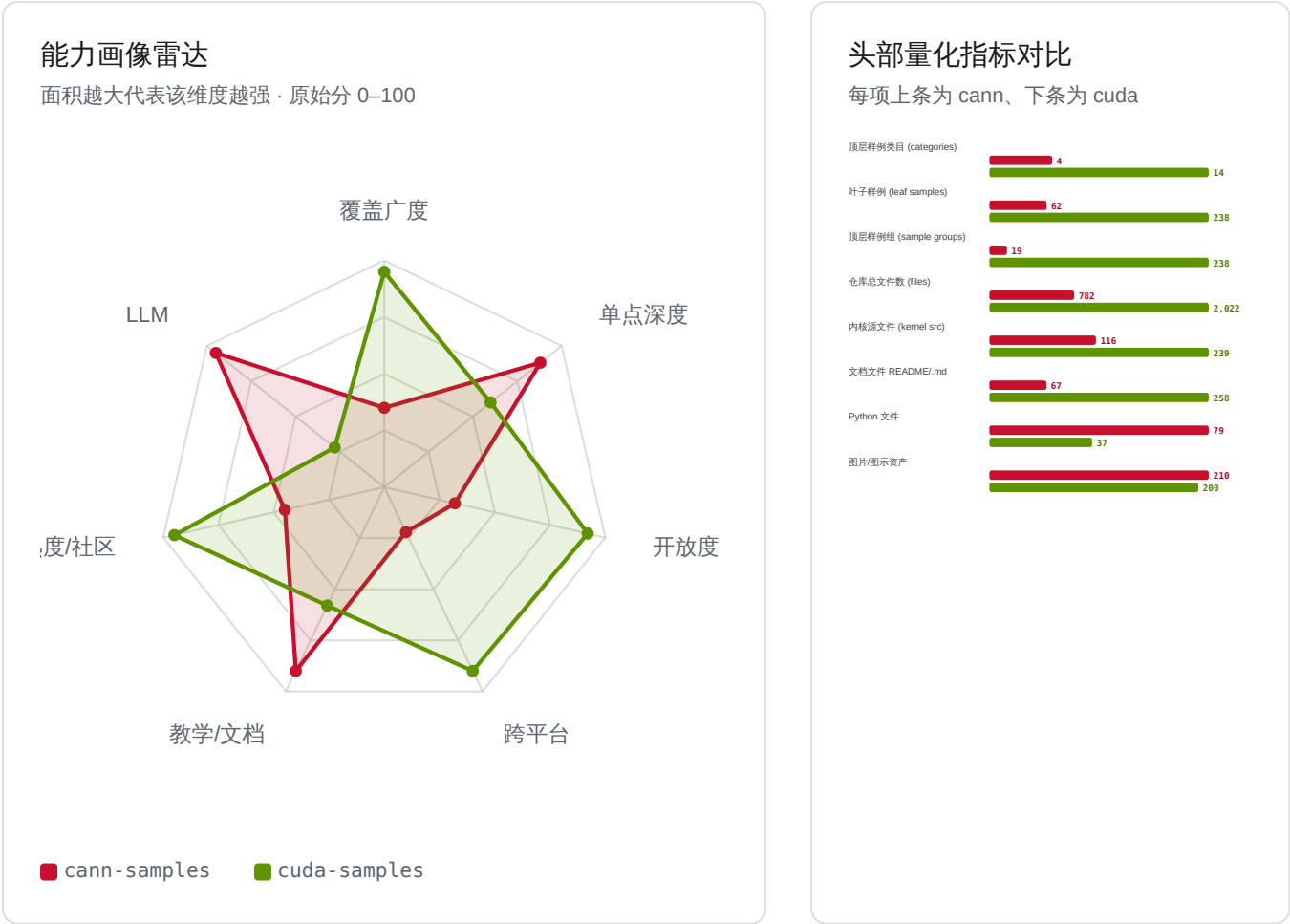
两个仓库名字都叫「samples」，本质却是两种不同的产物。**cuda-samples** 是一座「CUDA 工具链全景陈列馆」：以约 **238** 个自包含、结构一致的可运行样例，横扫从入门原语、并行算法、数值库（cuBLAS/cuFFT/cuSOLVER/NPP/CUB）、图形互操作（D3D/Vulkan/OpenGL）、经典 HPC/金融，到跨平台（Windows/Tegra/DriveOS）与 CUDA-Python 的极宽领域，并以 **BSD-3-Clause** 这一 OSI 认证的宽松许可自由分发。它的价值是广度与低门槛。

cann-samples 则是一部「昇腾算子调优知识库」：顶层仅 **4** 个类目、**19** 个样例组，却在 2_Performance 下用 9 个递进式 *_story 把矩阵乘、分组矩阵乘、MoE 派发/合并、融合注意力、KV-RMSNorm-RoPE、低比特（MX FP4/FP8、HiF8、INT8）量化等少数高价值 LLM 推理算子，从 0_naive 一路打穿到 7_fullload，配长文中文教程与 profiling 图。它的价值是深度与调优方法论——代价是受限于华为 AI 处理器场景的 **CANN OSL v2.0** 许可（非 OSI）与真机硬件门槛。

核心判断：二者非替代关系。若你要系统学习 **GPU** 生态广度、使用官方库、关心开源合规与跨平台，选 **cuda-samples**；若你要在昇腾上深挖单个 **LLM** 算子的极致性能与调优范式，选 **cann-samples**。在「开放度」这一单项上，**cuda-samples** 因宽松许可、全球托管与用途自由而明显领先；在「单点深度与中文调优文档」上，**cann-samples** 反超。

七轴能力画像与头部指标

下列七轴（0–100 分）是本报告的显式分析模型，由目录事实、覆盖广度/深度、许可条款与治理信号推导得出，非任一厂商官方指标。交互版报告允许读者对这七轴自定义权重、实时重算契合度。



如何读这张画像：cuda-samples 在覆盖广度、开放度、跨平台、成熟度四轴显著领先；cann-samples 在单点深度、教学/文档、LLM 算子聚焦三轴反超。两条多边形几乎互补——这正是「广度型工具链陈列」与「深度型算子知识库」的形状差异。

能力轴	CANN	CUDA	说明
覆盖广度 Breadth	35	95	领域/类目覆盖面
单点深度 Depth	88	60	单个算子/主题的打穿深度
开放度 Openness (许可)	32	92	许可宽松度与用途自由
跨平台 Portability	22	90	跨平台/跨硬件可移植性
教学/文档 Pedagogy	90	58	教程完整度与文档密度
成熟度/社区 Maturity	45	95	项目年龄/社区/连续性

能力轴	CANN	CUDA	说明
LLM 算子聚焦 AI-ops	95	28	大模型推理算子聚焦度

相同的编号骨架，不同的血肉

两仓都采用「数字前缀分层」的样例骨架，但 cuda 是扁平陈列（category/sample 两级到底），cann 是深层叙事式（category/story/{tutorials|recipes}/step/include，最深达 6–7 层）。cuda 靠样例数量铺开广度，cann 靠 story 内部的 **step/example** 展开深度。

cann-samples 目录树

4 顶层类目 · 深藏 story / recipes / tutorials

```
Samples/
├─ 0_Introduction/          入门 · 5 组
│   vector_add · matmul · npu_execution ...
├─ 1_Features/              特性 · 15 叶
│   └─ hardware_features/
│       hif8·simt·vector_function
│   └─ instruction_optimization/
│       mte2·unit_flag...
│   └─ memory_optimization/
│       full_load·bank_conflict
│   └─ system_optimization/
│       streamk·tail_rebalance
├─ 2_Performance/           性能调优 · 9 story
│   └─ matmul_story/
│       recipes(7)+tutorials(0→7)
│   └─ grouped_matmul_story/ recipes(4)
│   └─ moe_dispatch_and_combine_story/
│   └─ moe_init_routing_story/
│   └─
│       full_quant_fused_infer_attention_score_story/
│   └─ kv_rms_norm_rope_cache_story/
│       membase·regbase
│   └─ rms_norm_quant_story/ · simd_vf_story/
│   └─ scalar_story/
└─ 3_Utillities/            工具 · 1
    simulation-based-vf-profiling
```

cuda-samples 目录树

cpp/ 10 类目 (205) + python/ 4 类目 (33)

```
cpp/
├─ 0_Introduction/          46
│   vectorAdd·matrixMul
├─ 1_Utillities/            3   deviceQuery
├─ 2_Concepts_and_Techniques/ 32
│   reduction·scan
├─ 3_CUDA_Features/         24
│   graphs·tensorCore
├─ 4_CUDA_Libraries/        39
│   cuBLAS·cuFFT·NPP
├─ 5_Domain_Specific/       36
│   BlackScholes·nbody
├─ 6_Performance/           5   transpose
├─ 7_libNVVM/               9   NVVM IR /
│   ptxgen
├─ 8_Platform_Specific/     1   Tegra
├─ 9_CUDA_Tile/             10
│   tileMatmul·tileRope
│   python/
├─ 1_GettingStarted/        8
│   vectorAdd·deviceQuery
├─ 2_CoreConcepts/          20
│   cudaGraphs·prefixSum
├─ 3_FrameworkInterop/      2
│   PyTorch·TensorFlow
└─ 4_DistributedComputing/   3
    simpleP2P·multiGPU
```

类目语义对应关系

语义领域	CANN 对应	CUDA 对应	关系
入门原语	0_Introduction	0_Introduction / py 1_GettingStarted	对齐
并行算法/概念	— (仅 simd_vf 含 reduce)	2_Concepts_and_Techniques	cuda 独有
硬件/指令/访存特性	1_Features (4 子类)	3_CUDA_Features (部分)	cann 更细
数值库	— (仅 ops-tensor 子模块)	4_CUDA_Libraries	cuda 独有
领域应用/图形/金融	—	5_Domain_Specific	cuda 独有
算子性能调优	2_Performance (9 story)	6_Performance (5 样例)	cann 独深
LLM 推理算子	2_Performance (MoE/FIA/RoPE)	— (仅 9_CUDA_Tile 孤立原语)	cann 独有

语义领域	CANN 对应	CUDA 对应	关系
底层 IR / 编译	—	7_libNVVM	cuda 独有
Python / 框架互操作	— (Python 仅校验脚本)	python/ 3_FrameworkInterop	cuda 独有
跨平台/嵌入式	—	8_Platform_Specific (Tegra)	cuda 独有

结构哲学差异：cuda 是「字典型」仓库——每个样例自包含（.cu +README+CMakeLists）、结构高度一致，配全局 Common/（33 项 helper 头文件）与集中式 run_tests.py，适合「查 X 功能的最小示例」。cann 是「教程型」仓库——无全局 common，共享代码就近放在各 story 内，结构因样例而异（有的 tutorials/recipes 双分支、有的按 membase/regbase 分硬件路径），适合「学单算子从朴素到极致的优化过程」。

「真开源」与「源码可得」之间

这是全对比中最具决策分量的一节。二者「开放」的含义并不相同：cuda-samples 是 OSI 认证的宽松开源；cann-samples 是限定用途、可撤销的源码可得（source-available）许可。

许可证逐条对照

直接研读两仓 LICENSE 全文

许可名称

cann：CANN Open Software License Agreement v2.0

cuda：BSD 3-Clause（+ CUDA EULA 引用）

用途限制

cann §2.1/§3.1：仅可为搭载华为 AI 处理器的系统开发/分发，禁止服务其他厂商处理器

cuda：无任何用途或硬件限制，可商用可分发

可撤销性

cann §2.1 明写 revocable；§5 华为可随时终止、§5.2 授权 void ab initio（自始无效、须删除）

cuda：永久授权，无终止条款

专利/IP 反制

cann §5.1(b)(ii)：你若指控 Software 侵犯你 IP，华为可终止授权

cuda：无任何报复机制

OSI 认证

cann：否 cuda：是

开放度记分卡

逐项占优计分

1

2

9

开放度占优：cuda 9 · 持平 2 · cann 1（共 12 项）

cuda 定位

面向全体 CUDA 开发者的工具链特性陈列——「展示 CUDA Toolkit 能做什么」。广、浅、可运行、可自由取用。

cann 定位

面向昇腾算子工程师的实战调优知识库——「教你把某算子在昇腾上打到极致」。窄、深、重文档、限昇腾用途。

开源开放度记分卡（逐条）

评估维度	CANN-SAMPLES	CUDA-SAMPLES	占优
许可证类型	CANN OSL v2.0（受限）	BSD-3-Clause（宽松）	cuda
OSI 认证开源	否	是	cuda
使用范围限制	仅限为华为 AI 处理器开发/分发	无用途限制	cuda
许可可撤销 / 专利反制条款	可撤销 + 专利诉讼即终止	无（永久授权）	cuda

评估维度	CANN-SAMPLES	CUDA-SAMPLES	占优
衍生作品分发	须随附本协议、保留声明、限昇腾用途	保留版权声明即可，任意用途	cuda
主托管平台	GitCode（中国）	GitHub（全球）	cuda
GitHub 角色	只读镜像 sync/upstream/master	原生开发主仓	cuda
贡献流程	SIG + Committer/Maintainer 门禁，620 行 CONTRIBUTING	Fork + PR，pre-commit.ci	持平
功能性 CI 门禁	run_ci_functional.py（真机功能测试）	run_tests.py + test_args.json + pre-commit	持平
文档语言 / i18n	中文优先	英文 / 全球	cuda
硬件可得性	需 Ascend 910B/C 或 950 + 指定 Toolkit	任意 NVIDIA GPU	cuda
领域文档/调优知识库深度	逐步教程 + 性能建模 + profiling 图	以可运行样例为主，文档较简	cann

合规要点：BSD-3 仅要求「保留版权声明、不得以 NVIDIA 名义背书」，此外无任何限制、授权永久。CANN OSL v2.0 则把授权范围硬绑定到华为 AI 处理器，并保留在违约或专利诉讼时随时收回授权、要求删除衍生作品的权利。因此在「可自由分发 / 可用于非昇腾平台 / 长期法律确定性」上，cuda-samples 明显更开放；这也是选型时不可忽视的法律维度。

谁覆盖得更广，谁挖得更深

领域热力矩阵量化两仓的覆盖投入（无 / 基础 / 覆盖 / 深耕）。规模上 cuda 约为 cann 的 8 倍（238 vs 62 叶子样例、189 `.cu` vs 116 `.asc`），但 cann 在少数 LLM 算子上的纵深为 cuda 所无。

领域	CANN	CUDA
入门 入门原语 (vector add / hello / template)	深耕	覆盖
入门 设备/拓扑查询 (device / topology query)	基础	覆盖
通算 并行算法原语 (reduction / scan / sort / histogram)	基础	深耕
计算 矩阵乘 / GEMM	深耕	覆盖
LLM 注意力 / LLM 推理算子 (FIA / MoE / RoPE / RMSNorm)	深耕	基础
LLM 量化 (MX FP4/FP8 · HiF8 · INT8)	深耕	基础
库 数值库 (BLAS/FFT/SOLVER/RAND/CUB/Thrust/CCCL)	—	深耕
库 图像/信号处理 (NPP · JPEG · DCT · 滤波)	—	深耕
图形 图形互操作 (D3D11/12 · Vulkan · OpenGL)	—	深耕
HPC 金融/科学 HPC (BlackScholes · nbody · FDTD · Monte Carlo)	—	深耕
系统 执行模型 (Graphs · Streams · CDP · IPC · P2P)	基础	深耕
性能 性能调优方法论 (逐步优化实践)	深耕	基础
硬件 硬件微架构特性 (SIMD/SIMT · bank conflict · 多缓冲)	深耕	基础
系统 多卡/分布式 (multi-GPU · MPI · shmem)	基础	覆盖
生态 Python / 框架互操作 (PyTorch / TF / cuda-python)	—	覆盖
工具 仿真 / Profiling 工具	覆盖	基础
平台 跨平台/嵌入式 (Windows · Tegra · DriveOS)	—	深耕

公平性说明：cann 在「注意力 / LLM 推理算子」上的深耕（融合 RMSNorm+RoPE+KV-Cache、MoE 派发/合并、全量化 FIA）确为 cuda 所无；但非零重叠——cuda 在 `cpp/9_CUDA_Tile` 提供了孤立原语 `tileRope`（RoPE 前向）、`tileLayerNorm`、`tileBmm`。故矩阵中 cuda 该项记「基础」而非「无」：它有零件，但没有 cann 那种把整条算子链打穿的融合调优 story。

cuda 类目分布

cpp 205 + python 33 = 238

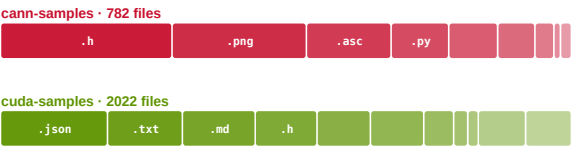
CUDA 类目	样例
0_Introduction	46
1_Utilities	3
2_Concepts_and_Techniques	32
3_CUDA_Features	24
4_CUDA_Libraries	39
5_Domain_Specific	36
6_Performance	5
7_libNVVM	9
8_Platform_Specific	1
9_CUDA_Tile	10
py 1_GettingStarted	8
py 2_CoreConcepts	20
py 3_FrameworkInterop	2
py 4_DistributedComputing	3
合计	238

cann 类目分布

展开 recipes/tutorials/variants ≈ 62

CANN 类目	样例
0_Introduction	5
1_Features	15
2_Performance	41
3_Utilities	1
合计	62

文件构成对比



一个是「样例库」，一个是「教科书」

同样叫 samples，开发体验却不同。cuda 每个样例是一段聚焦的可运行程序；cann 每个 story 是一部逐步教程，从 0_naive 优化到 7_fullload，配性能建模文档与 profiling 图。

cann：递进式 story / recipe

- 逐步教程：matmul_tutorials
0_naive → 1_pingpong → 2_block_swat → ...
→ 7_fullload，每步一个优化点
- 多配方 recipe：同一算子的
MXFP4/MXFP8/HiF8/INT8 多量化实现并列
- 重文档：67 篇 README/.md (均值 ~10.6KB 长文) + 210 张 tiling/流水/profiling 图
- 构建门槛高：cmake -DNPU_ARCH=dav-3510/2201 + ops-tensor 子模块 + 指定 Toolkit 版本
- Python 仅作数据生成/精度校验脚本，非可运行样例
- 硬件门槛：需 Ascend 910B/C 或 950 真机，特性样例挑架构

cuda：聚焦式一样例一特性

- 单点可运行：一个样例演示一个 API/特性，读完即跑
- NVRTC 变体：9 个 *_nVRTC 目录专教运行期编译 (JIT)
- 跨平台 CMake：Linux / Windows / Tegra 交叉 / DriveOS 一套构建
- 自动化回归：run_tests.py + test_args.json 逐样例参数化跑测
- 真 Python 样例：cuda-python + PyTorch/TF 自定义算子互操作
- 硬件门槛低：任意 NVIDIA GPU 即可，README 精简 (均值 ~2.3KB 卡片)

文档密度是最直观的分野：两仓 README 总量相近 (各约 585KB)，但 cuda 摊在 258 篇里、单篇均值仅 ~2.3KB (模板式卡片)；cann 集中在 67 篇里、单篇均值 ~10.6KB (长文教程)，单篇最大 64KB。cann README 含约 7.6 万汉字与 184 张图；simd_vf_story 单篇 README 达 1644 行、matmul_story 1180 行并配 8 级递进教程。

仅 **cann** 独有

- 逐步性能调优教程：matmul_tutorials 0_naive → 7_fullload (Ping-Pong / SWAT / UnitFlag / Half-L1 / 全载)
- LLM 推理算子专题 story：MoE dispatch/combine、MoE init routing、Fused-Infer-Attention-Score、KV-RMSNorm-RoPE-Cache
- 前沿低比特量化 recipes：MXFP4 / MXFP8 / HiFloat8 / INT8 及 Weight-NZ
- 昇腾微架构特性样例：HiF8、SIMT、Vector Function、L1 bank conflict、N-buffer、MTE2 preload、Unit Flag
- 面向 NPU 的访存/指令/系统三层优化方法 (StreamK、尾轮均衡、滑窗自适应模板)
- 基于仿真的 VF Profiling 工具链 (simulation-based-vf-profiling)
- 结构化社区治理：SIG ops-basic、Committer/Maintainer 分级评审、真机 CI 功能门禁

仅 **cuda** 独有

- 完整数值库样例：cuBLAS / cuFFT / cuSOLVER / cuRAND / NPP / nvJPEG / CUB / Thrust / CCCL 3.3
- 图形互操作：Direct3D 11/12、Vulkan、OpenGL 全套渲染 + CUDA 互操作
- 经典 HPC / 金融：BlackScholes、二项式期权、Monte Carlo、nbody、FDTD3d、Mandelbrot
- 并行算法教科书样例：reduction、scan、radix/merge sort、histogram、卷积、协作组
- CUDA 执行模型全景：CUDA Graphs、条件节点、Streams、动态并行(CDP)、IPC、P2P、Tensor Core GEMM
- 跨平台：Windows / Linux / Tegra 交叉编译 / DriveOS 汽车平台
- CUDA-Python 可运行样例 + PyTorch / TensorFlow 自定义算子互操作 + 新 CUDA Tile 编程模型
- 自动化测试：run_tests.py 驱动全样例回归，test_args.json 逐样例参数化

全球 PR 社区 vs 结构化 SIG 门禁

两仓代表两种开源运营范式：cuda 是 GitHub 原生、随 CUDA 工具链多年滚动演进的成熟全球生态；cann 是 GitCode 主场、GitHub 镜像、组织严密但年轻的中文优先社区。

cann-samples 生态		cuda-samples 生态	
主平台	GitCode (中国) · Issues/Discussions/SIG 均在此	主平台	GitHub (全球) · 原生 fork + PR 协作
GitHub	只读镜像 sync/upstream/master (cann-robot 自动同步)	版本	随 CUDA Toolkit 发布 (本次 v13.3) · CHANGELOG 上溯 CUDA 9.2/10.0
治理	SIG ops-basic · Committer /lgtn → Maintainer /approve	治理	NVIDIA 维护 · 外部 Fork + PR · 94 行 CONTRIBUTING
贡献规范	620 行 CONTRIBUTING (A/B/C 三种 Sample 模板) + OAT.xml 许可声明强制	CI 门禁	pre-commit.ci 格式化 (clang-format + ruff) + run_tests.py 回归
CI 门禁	run_ci_functional.py 真机功能测试	依赖	CPM 拉取 CCCL (pin v3.3.3) · 更去中心化
依赖	git 子模块 ops-tensor / shmem (钉死于 gitcode) + 45KB 第三方声明	成熟度	多年沉淀 · CHANGELOG 覆盖 CUDA 10.x → 13.3 · 数十版本连续
成熟度	年轻 · 镜像分支约 50 提交 (2026-05 → 07) · 约 21 位作者 (多为华为)	语言	英文 / 全球
语言	中文优先		

cann 生态强项：结构化治理与真机 CI 功能门禁、极深的中文调优文档、面向 AI 大模型算子的前沿覆盖 (MX 量化、MoE、注意力)。强制兼容性声明 (硬件/CANN 版本/shape 约束) 保证可复现。

cuda 生态强项：宽松许可带来的低法律摩擦、全球社区可达性、跨平台与数值库的极致广度、随工具链长期演进的成熟度、集中化自动测试。

什么时候选哪一个

二者不是替代关系，而是服务于不同目标、不同硬件栈的两种资源。

选 CANN-SAMPLES 当你...

攻昇腾算子深度

- 在 Ascend 910B/C 或 950 上做高性能算子开发/调优
- 需要 LLM 推理算子（MoE、融合注意力、RoPE+RMSNorm KV Cache）的工程实现范式
- 需要 MX/HiF8/INT8 低比特量化矩阵乘的完整 recipe
- 想系统学习「访存/指令/系统」三层逐步优化方法论
- 接受受限昇腾用途的许可与真机门槛，看重中文深度文档

选 CUDA-SAMPLES 当你...

学 CUDA 生态广度

- 要系统入门/概览 CUDA 全栈能力与 API 特性
- 需要 cuBLAS/cuFFT/cuSOLVER/NPP/CUB/Thrust 等库的可运行示例
- 做图形互操作（D3D/Vulkan/GL）或经典 HPC/金融计算
- 关心开源合规、可商用、可分发与跨平台（Windows/Tegra/DriveOS）
- 想要低硬件门槛（任意 NVIDIA GPU）与全球社区支持

一句话总览

维度	CANN-SAMPLES	CUDA-SAMPLES
一句定位	昇腾算子调优知识库	CUDA 工具链特性陈列馆
取胜之道	深度 · 调优方法论 · 中文文档	广度 · 低门槛 · 开源合规
规模	4 类目 / 19 组 / ≈62 叶 / 782 文件	14 类目 / 238 叶 / 2,022 文件
许可	CANN OSL v2.0（受限 · 非 OSI）	BSD-3-Clause（宽松 · OSI）
硬件门槛	特定 Ascend 真机 + 指定 Toolkit	任意 NVIDIA GPU
最适合	昇腾算子工程师 / LLM 推理优化	CUDA 学习者 / 生态应用开发者

全量计数与对抗式验证

A · 头部量化指标

指标	CANN	CUDA	口径备注
顶层样例类目 (categories)	4	14	cann: 4 (Samples/0-3) ; cuda: 10 (cpp) + 4 (python)
叶子样例 (leaf samples)	62	238	cann 展开 recipes/tutorials/variants 计数 ; cuda 205 cpp + 33 python
顶层样例组 (sample groups)	19	238	cann 顶层可见样例目录数 19 , 深藏于 story/recipes
仓库总文件数 (files)	782	2,022	
内核源文件 (kernel src)	116	239	cann .asc 116 ; cuda .cu 189 + .cuh 50
文档文件 README/.md	67	258	
Python 文件	79	37	cann: 数据生成/校验脚本 ; cuda: 可运行 CUDA-Python 样例
图片/图示资产	210	200	cann .png184+.svg26=210 ; cuda png20+jpg28+ppm54+pgm21+bmp12+gif6+...

B · 文件类型构成

CANN · 扩展名	数量	CUDA · 扩展名	数量
.h 头文件	235	.json 测试参数	379
.png 图片	184	.txt	265
.asc 内核源	116	.md 文档	258
.py 脚本	79	.h 头文件	219
.md 文档	67	.cu 内核源	189
.txt	51	.cpp	189
.svg	26	.hpp	106
.sh	9	.cuh 内核头	50
其它	15	.py 样例	37
		图片/数据	168
		其它	162

C · 对抗式核验日志

下列关键论断由独立子代理对两仓工作树执行反证式核验（尝试推翻而非确认），结论如下。

论断	判定	核验结论与修正
样例与类 目计数	已 证 实	cuda cpp=205（10 类目）、python=33（4 类目）、合计 238；cann 顶层类目 4、顶层样例组 19（0_Intro 5 / 1_Features 4 / 2_Performance 9 / 3_Uutilities 1）；matmul_recipes 7、grouped_matmul_recipes 4、matmul_tutorials 8 步。均逐目录枚举核对，精确匹配。
内核 源与 扩展 名	已 证 实	cann：.asc=116、.h=235、无 .cu/.cuh；cuda：.cu=189、.cuh=50、.h=219、无 .asc。 .asc 仅见于 cann（Ascend C 内核），.cu/.cuh 仅见于 cuda。四项计数精确。
许可 证性 质	已 证 实	cann LICENSE 第1行 = CANN OSL v2.0；§2.1 授权限「systems with Huawei AI Processors」、明写 revocable；§3.1 禁止服务其他处理器；§5.1(b)(ii) 专利/IP 诉讼即终止、§5.2 授权 void ab initio；非 OSI。 cuda LICENSE = 逐字 BSD-3-Clause（NVIDIA 2022）+ CUDA EULA 引用；OSI 认证宽松。
托管 与 CI	部 分修 正	均确认：cuda GitHub 原生、CHANGELOG 按 CUDA 版本、pre-commit.ci、run_tests.py；cann GitCode 主场、GitHub 镜像（.ci/mirror_update_time.txt）、run_ci_functional.py 功能门禁。修正：#435 为 v13.3 release PR（下界），线上实际更高；cuda 本地树无 .github/ Actions，CI 即 pre-commit.ci 外部服务；cann 的 CI 清单 tests/ci_functional_test.yaml 已被移除。
领域 独占 性	部 分修 正	cuda 独占（cann 全无）：图形互操作 D3D/Vulkan/GL、经典 HPC/金融、数值库 cuBLAS/cuFFT/cuSOLVER/NPP/CUB；确认无误。cann 独占（cuda 全无）：MoE 派发/合并、融合注意力 FIA、KV-RMSNorm-RoPE、MX 量化矩阵乘；确认。唯一重叠修正：cuda 在 cpp/9_CUDA_Tile/tileRope 提供孤立 RoPE 前向（另有 tileLayerNorm/tileBmm），故「cuda 完全不涉及 RoPE」为过度陈述——但 cann 的融合调优深度仍为 cuda 所无。

D · 方法与免责声明

对比对象	cann-samples sync/upstream/master；NVIDIA/cuda-samples master Release v13.3。快照 2026-07-09。
计数口径	文件数/类目/样例均由 find/ls 直接枚举（排除 .git）。cuda 叶子样例=cpp 205 + python 33=238；cann 叶子样例展开 recipes/tutorials/variants ≈62，顶层样例组 19。
能力评分	七轴 0–100 为显式分析模型，供交互版重加权，非厂商官方指标。
局限	cuda-samples 为浅克隆，历史以 PR 编号与 CHANGELOG 佐证；领域深度分级含主观成分。

本报告为独立技术对比分析，非任何厂商官方文档；商标与许可条款归各自权利人所有。数值以快照日两仓实际内容为准。